1 1 4 2 7 2

Dynamic skill activation (§ . .)

Keep the ACTIVE skill surface at the minimum needed right now. Everything else stays discoverable and one command away.

Pruning is the DEFAULT (operator decision, 2026-09-07): `session-init` demotes every non-core skill so a session pays only for `core`. It **deletes nothing** — see Pruning below for why that is not a §11.4.122 silent removal, and for the opt-out.

Why

Measured 2026-09-07, Claude Code 2.1.263, on a 955-skill pool:

render mode	bytes	~tokens per turn
name-only (observed)	29,153	~7,288
name + `description:` (observed for hot-loaded entries)	101,279	~25,319

Byte counts are exact. Token figures are **bytes/4 estimates** — there is no tokenizer on the measuring host, and they are labelled as estimates per §11.4.6.

Platform behaviour this is built on (PROVEN, not assumed)

behaviour	result	how it was established
activation hot-loads mid-session	**YES**	created a skill dir; it appeared in the running session with no restart
deactivation hot-unloads	**NO**	deleted the dir; the same session still invoked it from cache
`permissions.deny: ["Skill(x)"]` removes x from the rendered list	**NO**	a fresh headless session with the rule active still listed `x` — it blocks invocation, saves zero tokens

Consequence: the saving is a **minimal session-start baseline plus on-demand growth**. Deactivation takes effect at the NEXT session start. Any claim that deactivating frees tokens in the live session would be a bluff.

Use

```
# from anywhere, in either mode (with or without the constitution submodule):
~/.claude-shared/bin/skill-activate list          # full catalogue, [*]=active
~/.claude-shared/bin/skill-activate activate NAME # hot-loads NOW
~/.claude-shared/bin/skill-activate deactivate NAME
~/.claude-shared/bin/skill-activate status        # active set + byte budget
~/.claude-shared/bin/skill-activate prune         # demote non-core to the attic
~/.claude-shared/bin/skill-activate session-init  # prune + raise the core set
```

Agents reach the same thing through the always-core `skill-catalog` skill, so a deactivated skill is never unfindable (§11.4.102(B) keeps binding).

Pruning (the default)

`session-init` prunes first, then raises `core`. Measured on this project 2026-09-07:

	active skills	name-only render	name+description render
before	19	422 B (~105 tok/turn)	3,622 B (~905 tok/turn)
after	3	65 B (~16 tok/turn)	1,336 B (~334 tok/turn)

Byte counts are exact; token figures are **bytes/4 estimates** — there is still no tokenizer on the measuring host (§11.4.6).

Why this is not a § . . silent removal

§11.4.122 forbids removing an end-user **capability** without asking. Pruning removes no capability, because all three properties that make it reversible and visible hold by construction:

1. **Listed** — `skill-catalog` is always-core, so every pruned skill still appears in `skill-activate list`. Nothing becomes unfindable.
2. **Activatable** — `skill-activate activate NAME` brings it straight back, hot-loading into the running session.
3. **Not deleted** — a symlink loses only the *link* (its target is untouched); a real directory is **moved** to the attic (`.claude/.skill-activation/pool/`), never removed. Zero bytes destroyed.

A pruned skill is **dormant and one command away**, not gone. That is materially different from dropping a component from a build, which is what §11.4.122 exists to prevent. Do not "fix" pruning back out on §11.4.122 grounds without re-reading this.

The attic sits inside `.claude/`, which is already gitignored, so demotion changes nothing about a file's version-control status (§11.4.30 hygiene, no git-status noise).

Refusals are loud, never silent (§ . .)

Prune refuses and leaves the skill **active** rather than risk it when: another live session holds a claim (§11.4.119/§11.4.176 refcounting); an attic entry of that name already exists (never clobber preserved content, §9.2); the move fails or lands unreadable (the move is reverted); or a symlink's name resolves in no pool source and cannot be made re-activatable.

Opt-out

mechanism	scope	effect
`HELIX_SKILL_PRUNE=0`	one session	pruning off (env wins over manifest)
`HELIX_SKILL_PRUNE=1`	one session	pruning on, overriding the manifest
`prune: false` in the manifest	one project	pruning off

The escape hatch is deliberate: a mechanism with no way out gets worked around destructively — someone stops running `session-init` at all, and the surface grows back with no gate watching it.

Manifest (consumer DATA — § . .)

`<project>/.helix/skill-manifest.yaml`, else the shipped `skill-manifest.default.yaml`:

```yaml
schema_version: 1
core:                # always active — every entry is paid for EVERY turn
  - skill-catalog
  - action-prefix-system
sources:             # searched for <name>/SKILL.md; never modified
  - constitution/skills
  - .helix/skills
  - ~/.agents/skills
```

Both modes (REQ-NOTE-)

Resolution order, first hit wins: `$HELIX_SKILL_ENGINE` → in-project `constitution/` → any ancestor's → `$HELIX_CONSTITUTION_DIR` → `~/.claude-shared/constitution-path` → documented host fallback → **honest degradation**. Mode C exits **3** (distinct from 1 = operation failed) and names what is unavailable, why, where it looked, and that the existing static skill set is untouched — never a silent no-op.

Guards

- `skill_budget_gate.sh` — `CM-SKILL-SURFACE-BUDGET`. Budgets BOTH render modes, so an upstream switch to description-eager rendering fails LOUDLY. Limits: `HELIX_SKILL_BUDGET_NAME_BYTES` (4096), `HELIX_SKILL_BUDGET_DESC_BYTES` (24576).
- `test_skill_activation.sh` — 42 integration checks, 7 of them paired §1.1 mutations that must FAIL when the invariant is broken. The prune mutations run against a **copy** of the engine, never the live tree (§11.4.84 quiescence — this checkout is shared with other agents).

Known limitations (§ . .)

- Sessions sharing ONE config dir share one claim identity; override with `HELIX_SKILL_SESSION` to separate them.
- Liveness is proven by reading `/proc/<pid>/environ`. Where `/proc` is absent (macOS) liveness is reported as *cannot determine*, never guessed.

- A skill directory must contain `SKILL.md` (uppercase). Four constitution skills currently use lowercase `skill.md` or have none, so the platform cannot discover them: `media-validator`, `scheduled-work-queue`, `session-sync`, `multitrack`. Reported, not silently renamed (§11.4.124).
- Demotion makes a directory relocation-safe for **symlinks** by re-pointing relative links at their current absolute target (same file, new link text). A directory whose content depends on its own path some *other* way (a relative path baked into a file's text) is not repaired by that; the post-move readability check is the backstop and refuses + reverts rather than leaving a broken skill behind. Found live 2026-09-07: 5 `speckit-superspec-*` skills whose `SKILL.md` is a `../../../` symlink were refused on the first run, then demoted cleanly after the fix.
- Pruning's saving lands at the next session start for entries the platform has already cached — deactivation does not hot-unload (proven above). Run `session-init` at session start to get the reduced surface for that session.